

Operation Accountability as a First-Class Verification Boundary for Machine-Actionable Object Systems

Bin Zhang

Abstract

Runtime traces can help explain what an AI system did, but they do not by themselves provide a compact, independently checkable account of a single operation. This paper presents Execution Evidence and Operation Accountability Profile v0.1, a minimal artifact for one operation accountability statement in machine-actionable object systems, informed by digital-object concepts. The artifact serves as a minimal verification boundary for a bounded question rather than as a broad governance platform. It consists of a profile specification, a JSON Schema, a profile-aware validator exposed through `agent-evidence validate-profile <file>`, one valid example, three invalid examples that each break one main rule class, and one runnable metadata-enrichment demo that produces a validation report. The validator checks the bounded properties defined by the artifact: schema conformance, reference closure, consistency across policy/provenance/evidence links, and integrity digest recomputation. A fresh rerun reproduces the documented pass/fail behavior for the valid example, the three single-failure invalid examples, and the demo path. The contribution is intentionally narrow. It does not claim registry design, multi-agent orchestration coverage, formal deployment evidence, or broad platform outcomes. Instead, it offers an argument for treating operation accountability as a first-class verification boundary, with the minimal profile, validator, and demo serving as supporting evidence.

1. Introduction

Runtime traces and logs are useful for debugging operations in machine-actionable object systems, but they do not by themselves provide a compact, independently checkable account of a single operation. Related provenance and attestation lines show the value of structured records, explicit constraints, and downstream verification conditions when raw execution traces are not enough for later review [1], [2], [3]. The present paper addresses a deliberately bounded question: can one agent operation in a machine-actionable object system be expressed as a small accountability artifact that another party can review later without reconstructing the full runtime environment? Rather than proposing a general accountability platform, the paper advances a flagship claim: operation accountability should be treated as a first-class verification boundary, with the minimal implementation used here serving only as bounded supporting evidence.

The artifact centers on Execution Evidence and Operation Accountability Profile v0.1. It defines one fixed-shape operation accountability statement with explicit sections for actor, subject, operation, policy, constraints, provenance, evidence, and validation. The artifact also exposes a profile-aware validator through `agent-evidence validate-profile <file>`, provides one passing example and three controlled failing examples, and includes a runnable metadata-enrichment demo that ends in a validation report. For bounded background only, the paper draws on prior digital-object and FAIR Digital Object work rather than claiming to reproduce those broader architectures in full [4], [5], [6].

The contribution claimed here is intentionally narrow but not artifact-first. Its primary contribution is the problem definition and verification-boundary framing: operation accountability is treated as an independently reviewable boundary that can be expressed and checked in bounded form. The minimal profile,

validator pathway, controlled examples, and demo provide the supporting evidence vehicle for that argument. The paper does not attempt to define a general registry architecture, a multi-agent accountability system, a complete FDO interoperability layer, or a full cryptographic trust fabric. The value of the work is that it makes this verification boundary concrete and reviewable.

For the same case, adjacent forms such as logs, provenance-only representations, policy-only representations, and attestation-like records are best treated as partial capability holders rather than as strawmen. Logs can preserve runtime detail, provenance-only forms can preserve lineage, policy-only forms can preserve normative intent, and attestation-like records can preserve signed or constrained claims. What they do not necessarily bind together in one review object is the operation itself, the policy basis for that operation, the input and output references it touched, the emitted evidence, and the validation path a later reviewer can rerun. The point here is therefore not a superiority claim, but a same-case comparison: for the bounded question studied in this paper, those elements need to be held together within one independently checkable operation-accountability boundary.

2. Problem and Scope

2.1 Problem Definition

The problem addressed by the current artifact is not how to expand system capability or how to collect richer traces. It is the narrower problem of how to express one agent operation as an independently checkable accountability artifact. In the present paper, that artifact takes the form of one operation accountability statement governed by a minimal accountability profile. The statement is meant to answer a compact set of questions: who executed, which subject object was involved, what operation was invoked, which policy and constraints governed the action, how input and output objects

were referenced, what evidence artifacts were emitted, and what validation path a third party can apply.

This problem framing is intentionally tied to one statement and one operation. The target object of study is therefore not an open-ended execution trace but a fixed-shape accountability artifact that another party can inspect later without reconstructing an entire runtime environment. In that sense, the paper borrows vocabulary from broader digital-object, provenance, and attestation settings while narrowing them to a single statement-level review object [1], [3], [5].

2.2 Scope Boundary

The paper stays within a compressed boundary. The profile covers one statement, one operation, and one bounded validation surface. The example set is equally controlled: one valid profile instance and three invalid instances that each break one main rule class. The demo follows the same design choice by keeping to one metadata-enrichment path over one client note object under an explicit policy boundary.

Several common expansions are explicitly out of scope. The current contribution does not cover registry design, multi-agent orchestration or accountability composition, full FDO mapping, full cryptographic infrastructure, standard adoption claims, or external deployment claims. It also does not introduce new experiments beyond the documented example, validator, and demo path. The paper should therefore be read as a flagship argument about a minimal verification boundary rather than as a broad system or policy paper. Those larger directions remain active adjacent lines in digital- object, provenance, and attestation work, but they are not the claim of this paper [2], [6], [7].

3. Profile Design

3.1 Object Model

Execution Evidence and Operation Accountability Profile v0.1 defines a minimal JSON profile for one operation accountability statement. The top-level structure contains eleven sections: `profile`, `statement_id`, `timestamp`, `actor`, `subject`, `operation`, `policy`, `constraints`, `provenance`, `evidence`, and `validation`. Together, these sections capture a single bounded accountability record rather than an open-ended evidence model.

The design centers on four review questions. First, the statement must identify the actor, subject, and operation. Second, it must name the governing policy and the referenced constraints. Third, it must preserve cross-section linkage through subject, operation, policy, input, and output references. Fourth, it must expose enough integrity material for bounded recomputation of statement, reference, and artifact digests.

3.2 Compliance Conditions

A statement is compliant only if the profile identity matches the fixed v0.1 name and version, the JSON instance satisfies

the schema, internal references close correctly, input and output references resolve to the appropriate roles, policy/provenance/evidence linkages remain consistent, and the documented integrity digests recompute to the expected values. The validator therefore checks the bounded conditions defined by the profile, including schema conformance, reference closure, linkage consistency, and integrity digest recomputation.

The corresponding failure conditions are the symmetric negatives of those checks: missing required fields, schema shape violations, unclosed references, role mismatches, inconsistent cross-section linkage, or digest recomputation failure. This design gives the profile a narrow but explicit conformance surface. The emphasis on explicit validity conditions is closest in spirit to provenance-constraint and attestation specifications that separate representation from checking [2], [8], [9].

3.3 Bounded Design Choice

The narrowness of the profile is deliberate. The artifact is meant to support a bounded independent verification path for one operation accountability statement. It is not meant to serve as a general provenance model, a platform architecture, or a complete trust infrastructure. The design choice is therefore to keep the statement small enough to validate directly while still preserving the actor, subject, policy, evidence, and integrity links needed for review.

4. Validator and Demo Path

4.1 Validator Path

The verification path is implemented as a CLI validator invoked through `agent-evidence validate-profile <file>`. Its output is a validation report: machine-readable JSON with an `ok` flag, issue count, explicit error codes when validation fails, and a short summary line. This interface is central to the artifact because it turns the profile from a passive data format into a checkable review path.

The current evaluation surface includes one passing example and three controlled failing examples that each target one main rule class. For the valid example, the report is expected to return `ok: true` with zero issues. For the three invalid examples, the report is expected to return `ok: false` with one primary error surface each.

4.2 Controlled Example Set

The example set is intentionally small. The missing-required-field case breaks required field completeness and produces `schema_violation`. The unclosed-reference case breaks internal reference closure and produces `unresolved_output_ref`. The broken policy-link case breaks the evidence-to-policy linkage and produces `unresolved_evidence_policy_ref`. The regression tests confirm those error-code outcomes for the controlled invalid specimens.

This structure matters because it keeps the evaluator’s task simple. The paper does not claim broad coverage over many rule combinations. It claims that the current evaluation surface is intentionally limited to one passing example and three controlled failing examples, each targeting one main rule class. That bounded example design is consistent with artifact-review norms that prioritize documented, exercisable, and reviewable research objects over large benchmark surfaces [10].

4.3 Bounded Failure Taxonomy

The three controlled failing specimens do not exhaust the bounded failure surface defined by the current profile. They directly ground three current classes, while integrity recomputation remains part of the validator-defined surface without being broken out as its own standalone failing specimen.

Failure surface	What it checks	Current grounded status
schema shape	whether the statement keeps the required fixed-shape structure	grounded by <code>schema_violation</code>
reference closure	whether internal references close correctly across the statement	grounded by <code>unresolved_output_ref</code>
policy/evidence linkage	whether evidence links resolve to the governing policy path	grounded by <code>unresolved_evidence_policy_ref</code>
integrity recomputation	whether documented digests recompute to the expected values	validator-defined surface, but not currently broken out as a separate failing specimen

4.4 Demo Scenario

The demo implements one policy-constrained metadata-enrichment path. It loads one source object, runs a minimal profile precheck, applies one constrained operation, generates one operation accountability statement, validates it, and writes artifacts for review. The scenario constrains the operation to adding approved metadata fields while keeping the note body unchanged.

The demo is therefore a single illustrative walkthrough rather than a benchmark or deployment study. Its role in the paper is to show that the current profile and validator path can be exercised end to end on one bounded scenario and can emit the expected evidence and validation report.

5. Evaluation

The current evaluation is intentionally artifact-bounded. It does not introduce new datasets, external deployments, or broad comparative experiments. It is limited to one valid example, three single-failure invalid examples, and one single-path demo for Execution Evidence and Operation Accountability Profile v0.1. This evaluation posture follows established artifact-review norms for research objects whose value lies in documented and exercisable evidence rather than comparative performance [10].

5.1 Fresh Clean Rerun

A fresh local rerun was performed on 2026-04-07 against public artifact commit 80e7e78ab6cbd9befc24b56fbf9cdfabd99b5de in a new virtual environment created specifically for this check. The environment used Python 3.14.3 on macOS arm64. The minimal tested install path was the base editable install (`pip install -e .`) from the public artifact repository root, and no extra dependency set was required for the current minimal profile/validator/demo path.

The install sequence used in the clean rerun was:

```
python3 -m venv .repro-v0_1-20260407-full
.repro-v0_1-20260407-full/bin/python -m pip
install --upgrade pip
.repro-v0_1-20260407-full/bin/python -m pip
install -e .
```

The command list used for the evaluation rerun was:

```
.repro-v0_1-20260407-full/bin/agent-evidence
validate-profile examples/minimal-valid-
evidence.json
.repro-v0_1-20260407-full/bin/agent-evidence
validate-profile examples/invalid-missing-
required.json
.repro-v0_1-20260407-full/bin/agent-evidence
validate-profile examples/invalid-unclosed-
reference.json
.repro-v0_1-20260407-full/bin/agent-evidence
validate-profile examples/invalid-policy-link-
broken.json
.repro-v0_1-20260407-full/bin/python
demo/run_operation_accountability_demo.py
```

5.2 Observed Outcomes

The observed command outcomes matched the current expected behavior.

Command target	Expected exit	Observed exit	Observed exit
examples/minimal-valid-evidence.json	0	0	0
examples/invalid-missing-required.json	1	1	1
examples/invalid-unclosed-reference.json	1	1	1
examples/invalid-policy-link-broken.json	1	1	1
demo/run_operation_accountability_demo.py	0	0	0

The valid example therefore produced a passing validation report with zero issues. Each invalid example failed with one primary error code and one main broken rule class. The demo produced a passing summary line and emitted a minimal profile evidence artifact together with its validation report. Raw local logs were retained as internal reproducibility records.

5.3 Bounded Interpretation

This rerun supports a bounded claim only: the current package can be re-executed in a clean environment and can reproduce the documented pass/fail behavior for one valid example, three single-failure invalid examples, and one demo path. It does not establish broader coverage, deployment robustness, or cross-framework generality. Those questions remain outside the scope of the present artifact and are therefore left for future work.

6. Related Work

6.1 DOIP / Digital Object / FDO Foundations

Digital object work provides the broadest conceptual backdrop for the present paper. Kahn and Wilensky define a distributed digital object architecture around identification, repository access, and object-oriented service interactions [4]. DOIP 2.0 turns that lineage into an explicit protocol for client interaction with digital objects and the services that manage them [5]. More recent FAIR Digital Object work evaluates FDOs and linked data as distributed object systems and discusses interoperability and machine actionability at a much larger systems level [6].

These works matter because they establish that digital objects, protocols, and machine-actionable object ecosystems are already active research and standards lines. However, they operate at the level of object architectures, interoperability protocols, or distributed object system evaluation. The present paper does not propose another object architecture, another repository protocol, or a general FDO implementation strategy. Unlike DOIP, the Digital Object Architecture, and broader FAIR Digital Object evaluations, this paper only targets one operation accountability statement and a bounded validation path.

6.2 Provenance Models and Validation Constraints

The W3C PROV family provides the main reference point for provenance modeling and validity checking. PROV-Overview explains the family structure and locates PROV-DM as the core data model, alongside PROV-Constraints as the document that defines valid instances for implementors of validators [11]. PROV-DM defines a general model for entities, activities, and agents in provenance interchange [1]. PROV-Constraints then formalizes definitions, inferences, normalization, equivalence, and constraint-based validity checking for provenance records [1].

This line of work is important because it separates provenance representation from provenance validation and makes explicit that consistency conditions can be first-class specification material. That is the closest methodological comparison point for the current artifact path. However, the present paper does not claim a general provenance data model, a full interchange family, or a complete validator for open-ended provenance documents. Unlike PROV-DM and PROV-Constraints, this paper only targets one operation accountability statement and a bounded validation path.

6.3 Adjacent Verifiable Provenance / Attestation Frameworks

Software supply-chain security offers adjacent examples of verifiable provenance-like artifacts. in-toto describes a framework in which software supply-chain steps, actors, and ordering constraints are made transparent and verifiable [3], and the stable v1.0 specification defines the corresponding layout, link-metadata, and verification concepts in operational form [8]. SLSA provenance and build-provenance specifications likewise define verifiable provenance information for software artifacts and a predicate for describing how build artifacts were produced so that downstream consumers can verify them against expectations and, when possible, rebuild them [7], [9].

These frameworks are relevant because they show that structured provenance or attestation documents can be operationalized for downstream verification. Their problem setting, however, is the software supply chain, including end-to-end build or release integrity and trust in builder identities. The current paper does not define a supply-chain layout, a trust policy, or a builder-attestation ecosystem. Unlike in-toto and SLSA provenance, this paper only targets one operation accountability statement and a bounded validation path.

6.4 Artifact Evaluation and Reproducibility Norms in Computing

The current evaluation posture is best understood through artifact-review norms rather than through large experimental evaluation. ACM's artifact review and badging policy frames artifacts in terms such as documented, consistent, complete, exercisable, and results validated [10]. That language is directly useful here because the core research object is a runnable artifact consisting of a profile, a schema, a validator

command, a controlled example set, and one demo path that can be re-executed in a fresh environment.

This norm is a better fit for the present contribution than benchmark-oriented evaluation language. The value of the work is not comparative performance but a compact artifact that reviewers can inspect and exercise against explicit expected outcomes. Unlike general artifact evaluation guidance for broad computational results, this paper only targets one operation accountability statement and a bounded validation path.

7. Threats to Validity

7.1 Scope Narrowness

The most important validity threat is scope narrowness. The artifact validates one profile for one statement shape and demonstrates one single-path metadata enrichment scenario. This is appropriate for a minimal specimen, but it limits how far the paper can generalize beyond the documented path.

7.2 Evaluation Breadth

The evaluation remains intentionally small. It covers one valid example, three invalid examples, and one demo path. That is strong enough to support claims about internal coherence and bounded verification behavior, but it does not support claims about large-scale coverage, production reliability, or cross-framework deployment behavior.

7.3 Surrounding System Coverage

The profile explicitly avoids registry design, multi-agent orchestration, full FDO mapping, and a full cryptographic trust fabric. Reviewers could reasonably ask whether the artifact extends to those settings, but the current paper should answer that those questions are deferred rather than solved.

7.4 Historical Material Separation

The broader artifact history still contains earlier Execution Evidence Object and Agent Evidence Profile materials.

[1] L. Moreau and P. Missier, “PROV-DM: The PROV data model,” World Wide Web Consortium, W3C Recommendation, Apr. 2013. Accessed: Apr. 07, 2026. [Online]. Available: <https://www.w3.org/TR/prov-dm/>

[2] J. Cheney, P. Missier, and L. Moreau, “Constraints of the PROV data model,” World Wide Web Consortium, W3C Recommendation, Apr. 2013. Accessed: Apr. 07, 2026. [Online]. Available: <https://www.w3.org/TR/prov-constraints/>

[3] S. Torres-Arias, H. Afzali, T. K. Kuppusamy, R. Curtmola, and J. Cappos, “In-toto: Providing farm-to-table guarantees for bits and bytes,” in *28th USENIX security symposium (USENIX security 19)*, USENIX Association, Aug. 2019, pp. 1393–1410. Available: <https://www.usenix.org/conference/usenixsecurity19/presentation/torres-arias>

[4] R. Kahn and R. Wilensky, “A framework for distributed digital object services,” *International Journal on Digital Libraries*, vol. 6, no. 2, pp. 115–123, 2006, doi: [10.1007/s00799-005-0128-x](https://doi.org/10.1007/s00799-005-0128-x).

[5] DONA Foundation, “Digital object interface protocol specification, version 2.0,” DONA Foundation, Protocol Specification, Nov. 2018. Accessed: Apr. 07, 2026. [Online]. Available: https://www.dona.net/sites/default/files/2018-11/DOIPv2Spec_1.pdf

[6]

The current manuscript must keep those historical surfaces separate from the present v0.1 path to avoid terminology drift or overstatement about the scope of the contribution.

8. Conclusion

The paper makes a narrow but specific claim: operation accountability should be treated as a first-class verification boundary for machine-actionable object systems. The public artifact provides a reproducible minimal path for expressing and checking one operation accountability statement. That path is concretized through Execution Evidence and Operation Accountability Profile v0.1, its JSON Schema, the agent-evidence validate-profile <file> validator entry point, controlled examples, and one runnable metadata-enrichment demo.

The value of the artifact is not that it resolves broad system or policy questions. Its value is that it turns a frequently abstract discussion about execution evidence and operation accountability into bounded supporting evidence that can be inspected, validated, and critiqued against explicit validation behavior and a fresh rerun. On that bounded line, the contribution is the verification-boundary formulation and its evidence-bounded validation framing, with the minimal profile, validator path, controlled examples, and demo serving as supporting evidence rather than as ends in themselves. That reviewer-facing role is consistent with artifact-review language that values documented, consistent, complete, exercisable, and validated research objects [10].

Minimality is a design choice here, not a hidden deficiency. By keeping the package to one statement, one validator path, one controlled example set, and one runnable demo, the paper isolates a concrete verification boundary that reviewers can inspect and rerun without requiring a larger platform or deployment context. In this sense, the contribution is not a better logging surface but a bounded accountability artifact used to make an independent verification boundary reviewable.

S. Soiland-Reyes, C. Goble, and P. Groth, “Evaluating FAIR digital object and linked data as distributed object systems,” *PeerJ Computer Science*, vol. 10, p. e1781, 2024, doi: [10.7717/peerj-cs.1781](https://doi.org/10.7717/peerj-cs.1781).

[7]

SLSA Community, “Provenance.” Accessed: Apr. 07, 2026. [Online]. Available: <https://slsa.dev/provenance>

[8]

in-toto Authors, *In-toto specification*. 2023. Accessed: Apr. 07, 2026. [Online]. Available: <https://github.com/in-toto/specification/blob/v1.0/in-toto-spec.md>

[9]

SLSA Community, “Build: provenance.” Accessed: Apr. 07, 2026. [Online]. Available: <https://slsa.dev/spec/v1.2-rc2/build-provenance>

[10]

Association for Computing Machinery, “Artifact review and badging - current.” Accessed: Apr. 07, 2026. [Online]. Available: <https://www.acm.org/publications/policies/artifact-review-and-badging-current>

[11]

P. Groth and L. Moreau, “PROV-overview: An overview of the PROV family of documents,” World Wide Web Consortium, W3C Working Group Note, Apr. 2013. Accessed: Apr. 07, 2026. [Online]. Available: <https://www.w3.org/TR/prov-overview/>